

# Distributed Job Scheduler

## Design & Implementation Report

Laware Sujit Annasaheb

July 5, 2026

<https://github.com/sujit-al1809/job-scheduler>

## 1 Introduction

This report goes with my submission for the distributed job scheduler assignment. The system has four parts: a REST API built with FastAPI, a small scheduler process, worker processes that can be scaled horizontally, and a React dashboard. All of them talk to a single PostgreSQL database.

The one decision that shapes everything else is that PostgreSQL is also the queue. There is no Redis or RabbitMQ. Workers pull jobs directly from the jobs table using `SELECT ... FOR UPDATE SKIP LOCKED`, which guarantees a job can only be claimed by one worker at a time. I explain why I went this way in Section 8, along with the other trade-offs I made.

The rest of the report covers the architecture, the database schema, the job lifecycle, how concurrency and failures are handled, the API, the dashboard, and testing. Setup instructions are at the end.

Stack summary: Python 3.12, FastAPI, SQLAlchemy 2.0 (async) with Alembic migrations, PostgreSQL 16, JWT auth, croniter for cron parsing. Frontend is React 18 + TypeScript + Vite, with TanStack Query and Recharts. Tests use pytest against a real Postgres container.

### Where things are

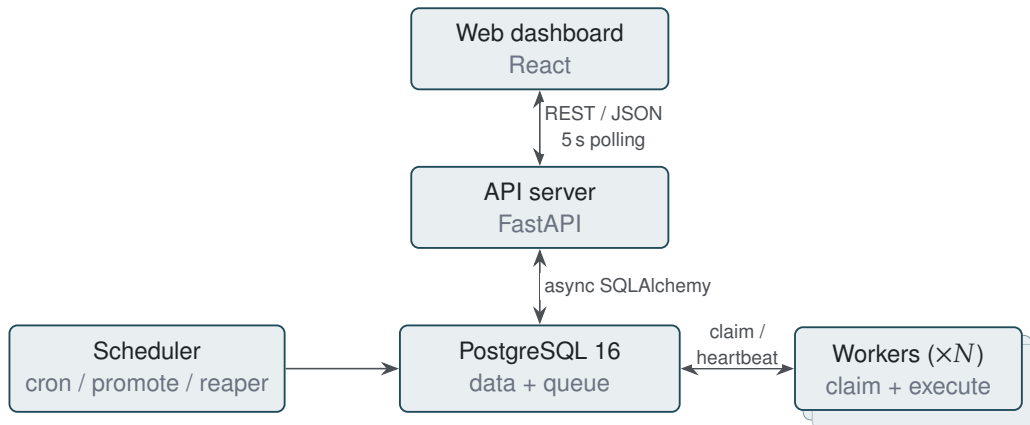
For quick reference against the brief, here is where each part is covered.

| Area                      | Section                                                            |
|---------------------------|--------------------------------------------------------------------|
| System architecture       | §2 (components, process split, why Postgres)                       |
| Database design           | §3 (schema, indexes, keys, normalisation, cascades)                |
| Job lifecycle             | §4 (state machine, transition rules)                               |
| Reliability & concurrency | §5 (atomic claiming, retries, DLQ, heartbeats, recovery, shutdown) |
| Backend & API design      | §6 (auth, validation, pagination, filtering, errors, endpoints)    |
| Frontend & UX             | §7 (dashboard views)                                               |
| Testing                   | §9 (unit + integration, concurrency suite)                         |
| Setup                     | §10; deliverables checklist in §11                                 |

## 2 Architecture

---

## 2.1 Components



The API is the control plane: auth, projects, queues, job submission, and all the read endpoints the dashboard needs. The scheduler owns anything time-based: it flips delayed jobs to ready when their `run_at` arrives, stamps out concrete jobs from cron templates, and requeues jobs stuck on dead workers. Workers do the actual execution.

The important property is that these processes never talk to each other. All coordination goes through database state. That is what makes it safe to run any number of workers (and even multiple schedulers) — there is no leader, no shared memory, nothing to elect.

## 2.2 Why Postgres as the queue

Three reasons, one cost.

First, enqueueing a job, updating its status, and writing its execution record all happen in the same ACID transaction. With a separate broker you get the classic dual-write problem: the broker and the database can disagree after a crash, and you need reconciliation logic. Here that problem does not exist.

Second, `FOR UPDATE SKIP LOCKED` gives correct concurrent dequeuing out of the box. Postgres has supported it since 9.5 and it is a well-known pattern for exactly this use case.

Third, it is one less piece of infrastructure to run, back up, and monitor.

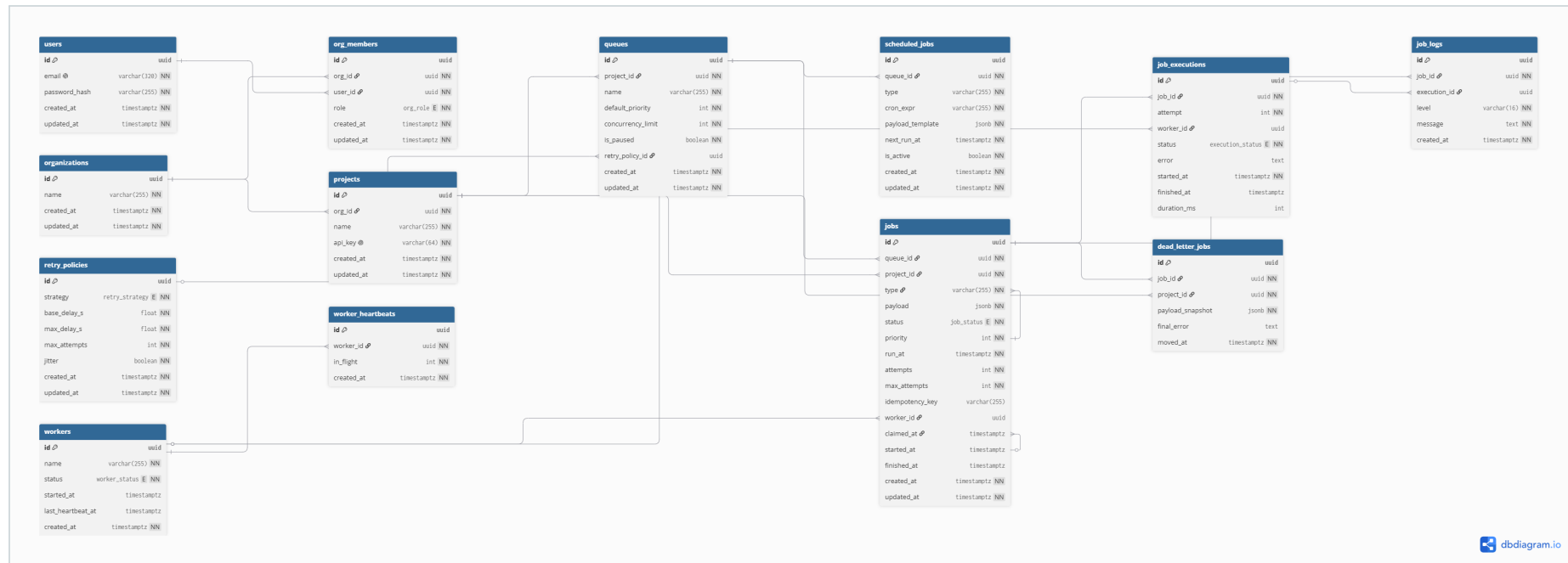
The cost is polling latency (workers poll every second by default rather than being pushed to) and a lower throughput ceiling than a dedicated broker. For a background-job system at this scale, both are acceptable. Section 8 covers the migration path if that ever changes.

---

## 3 Database design

### 3.1 Entity overview

The full column-level schema is in Figure 1 (next page, rotated to fit). Every relationship shown is a foreign key. Two of them are *behavioural* rather than structural and worth calling out: `jobs.worker_id` records the worker currently holding a job (nullable, cleared on release or worker deletion), and `scheduled_jobs` → `jobs` is the cron template materialising concrete job rows. Everything else is ordinary ownership. Deletion behaviour and the reasoning behind keys and indexes follow in the subsections below.



**Figure 1.** Full relational schema (13 tables). Generated from the DBML in docs/er-diagram.dbml via dbdiagram.io. PK and FK markers, column types, enum types, and every foreign-key relationship are shown.

## 3.2 Tables

| Table             | What it holds                                                                                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users             | Accounts. email is unique, passwords stored as bcrypt hashes.                                                                                                                                                                    |
| organizations     | Tenancy root. Every other resource is reachable from exactly one org, so authorisation reduces to scoping every query by the caller's org.                                                                                       |
| org_members       | Many-to-many between users and orgs, with a role column (owner/admin/member). Unique on (org_id, user_id). The role column is where RBAC would plug in.                                                                          |
| projects          | Groups queues, owns an API key. org_id FK, cascade on delete.                                                                                                                                                                    |
| retry_policies    | Retry config as data: strategy (fixed / linear / exponential), base_delay_s, max_delay_s, max_attempts, jitter. Queues point at one as their default; individual jobs can override.                                              |
| queues            | name (unique per project), default_priority, concurrency_limit, is_paused, retry_policy_id. Cascade from project.                                                                                                                |
| jobs              | The central table. queue_id, type, payload (jsonb), status (Postgres enum), priority, run_at, attempts, max_attempts, optional idempotency_key, worker_id (set null on worker delete), plus claimed/started/finished timestamps. |
| job_executions    | One row per attempt: job_id, attempt (unique together), worker_id, status, error, start/finish times, duration_ms. All metrics are computed from this table.                                                                     |
| job_logs          | Append-only log lines from handlers and from lifecycle transitions. job_id, nullable execution_id, level, message, created_at.                                                                                                   |
| scheduled_jobs    | Cron templates. cron_expr, payload template, next_run_at, is_active. These are never executed directly; the scheduler creates a real jobs row each tick.                                                                         |
| workers           | Worker registry: name, status (online / stopped / dead), started_at, last_heartbeat_at.                                                                                                                                          |
| worker_heartbeats | Heartbeat time series with an in_flight count. Used by the dashboard and by the dead-worker check.                                                                                                                               |
| dead_letter_jobs  | Jobs that exhausted their retries: FK to the original job, a payload snapshot, the final error, and moved_at. Separate table so the hot jobs table stays small and DLQ retention can be handled differently.                     |

## 3.3 Indexes

The index that matters most is a partial composite index on jobs:

```
CREATE INDEX idx_jobs_claim ON jobs (queue_id, priority DESC, run_at, id)
WHERE status = 'queued';
```

The claim query's ORDER BY matches this exactly, so dequeuing is an index range scan over only the ready rows, no matter how large the table gets. Making it partial keeps it small, since completed and dead jobs (the vast majority over time) never enter it.

The rest: jobs (queue\_id, status) for the dashboard's depth counters; a partial unique index on jobs (project\_id, idempotency\_key) where the key is not null, which is what enforces idempotent submission; a unique index on job\_executions (job\_id, attempt) so duplicate attempt rows are impossible; a partial index on scheduled\_jobs (next\_run\_at) where active, for the scheduler's scan; workers (last\_heartbeat\_at) for the reaper; and all foreign keys are indexed.

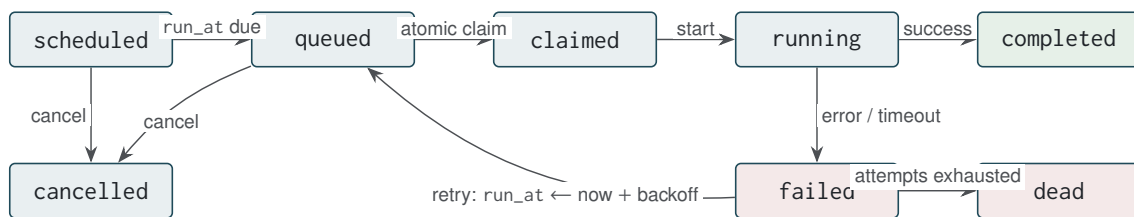
### 3.4 Keys, normalisation, cascades

Primary keys are UUIDs generated by the database. I picked UUIDs over serial integers because ids get created from three different processes and I did not want any coordination, and because they are safe to expose in URLs. All timestamps are `timestampz`; the `run_at` arithmetic depends on it.

The schema is in 3NF. Config (retry policies), definitions (queues, cron templates), instances (jobs), and history (executions, logs, heartbeats) are separate relations, and derived values like queue stats or success rates are always computed from `job_executions` rather than stored. The one denormalisation I allowed on purpose is `jobs.attempts`: it duplicates a count that exists in `job_executions`, but the retry decision happens on the hot path at failure time and I did not want an aggregate query there.

Deletes follow ownership: `org` → `projects` → `queues` → `jobs` cascade. `jobs.worker_id` is set-null instead, since execution history should survive a worker being removed, and `dead_letter_jobs` keeps its own payload snapshot for the same reason.

## 4 Job lifecycle



This covers every state the brief lists — `scheduled`, `queued`, `claimed`, `running`, `completed` — plus `failed`, `cancelled`, and the terminal `dead` state. “Dead letter queue support” is the `dead` state together with the `dead_letter_jobs` table (Section 5.1, retry subsection): a job that exhausts its retries lands there with its error and payload preserved.

Two implementation rules back this diagram. Every transition goes through one function, `services/job_state.py`, which checks the edge is legal, sets the right timestamps, and appends a `job_logs` row. Nothing else in the codebase writes `jobs.status`, so an illegal transition simply cannot happen. And retries are scheduled rather than held: a failed job goes back to `queued` with a future `run_at`. No worker ever sleeps while owning a job, so a queue full of backing-off jobs costs zero worker capacity. Cancellation is allowed from any state before `running`.

## 5 Concurrency and reliability

### 5.1 Claiming without duplicates

Workers claim batches with a single statement, run in one transaction:

```
UPDATE jobs SET status = 'claimed', worker_id = :worker_id, claimed_at = now()
WHERE id IN (
    SELECT j.id FROM jobs j
    JOIN queues q ON q.id = j.queue_id
    WHERE j.status = 'queued'
    AND j.run_at <= now()
    AND q.is_paused = false
```

```

ORDER BY j.priority DESC, j.run_at ASC, j.id ASC
LIMIT :batch_size
FOR UPDATE OF j SKIP LOCKED
)
RETURNING *;

```

FOR UPDATE locks the selected rows; SKIP LOCKED makes any concurrent worker skip rows that are already locked instead of blocking on them. So  $N$  workers polling the same queue split the ready jobs between them without ever waiting on each other, and a job cannot be claimed twice. This is tested directly: four claim loops racing over 200 seeded jobs must produce exactly 200 executions with no duplicates. The ORDER BY gives priority scheduling with FIFO tie-breaking and matches the partial index from Section 3.3.

## 5.2 Per-queue concurrency limits

At claim time, a queue is skipped if its count of claimed + running jobs has already reached its `concurrency_limit`. Under READ COMMITTED, two workers can race this check and briefly overshoot by a small amount, so the limit is soft. Making it hard would mean serialising all claims per queue, which defeats the point of having concurrent workers, so I documented the race instead of hiding it.

## 5.3 Retries and the dead letter queue

Retry behaviour comes from the retry policy attached to the queue (or overridden on the job). For attempt  $a$ , base delay  $b$  and cap  $d_{max}$ :

$$d_{\text{fixed}} = b \quad d_{\text{linear}} = b \cdot a \quad d_{\text{exp}} = b \cdot 2^{a-1}$$

with  $d$  capped at  $d_{max}$ , and multiplied by a random factor in  $[0.8, 1.2]$  when jitter is on. Jitter matters when a downstream dependency goes down: without it, every affected job retries at the same instant and hammers the dependency again.

When attempts run out, the job moves to dead and a snapshot goes into `dead_letter_jobs`. The DLQ is fully exposed in the API and dashboard — you can inspect entries, re-enqueue them with a fresh attempt counter, or delete them.

## 5.4 Heartbeats and crash recovery

Each worker updates `workers.last_heartbeat_at` and inserts a `worker_heartbeats` row (with its in-flight count) every few seconds. The scheduler runs a reaper: any worker silent for longer than the timeout is marked dead, and its claimed or running jobs are put back to queued with the attempt counter bumped and a log line explaining why.

The practical consequence: `kill -9` on a worker mid-job delays that job by at most one timeout window. It is never lost and never stuck. This makes the system at-least-once, not exactly-once, which is why handlers have to be idempotent — see the next subsection and decision 3 in Section 8.

## 5.5 Idempotency

Two layers. On submission, a client can pass an `idempotency_key`; the partial unique index turns a duplicate submit into a 200 response with the existing job instead of a second row, so client retries and double-clicks are safe. On execution, handlers receive (`job_id`, `execution_id`,

payload) and are written so that running twice does not double-apply side effects — upserts where possible, and external idempotency keys derived from the job id where not.

## 5.6 Graceful shutdown

On SIGTERM or SIGINT a worker stops claiming, gives in-flight jobs a drain window to finish, releases anything still unfinished back to queued, and marks itself stopped. So a normal deploy or scale-down loses nothing and leaves no orphans, without even needing the reaper.

## 6 API design

Everything is under `/api/v1` and needs a JWT bearer token, except registration, login and `/health`. Routers stay thin; the actual logic lives in a service layer that the worker and scheduler import too, so lifecycle rules cannot drift apart between processes.

Every list endpoint takes `limit/offset` plus filters (for jobs: `status`, `queue`, `type`, `created before/after`) and returns `{items, total, limit, offset}`. Errors, including validation errors, all come back in one shape:

```
{ "error": { "code": "QUEUE_NOT_FOUND", "message": "...", "details": { ... } } }
```

Requests for resources in another org return 404 rather than 403, so callers cannot probe for existence. Every request gets a correlation id and structured JSON logging, and the worker logs with the same format, so one job can be traced across processes. FastAPI serves interactive docs at `/docs`; the exported `openapi.json` is committed under `docs/`.

| Method                     | Path                                               | Description                                                             |
|----------------------------|----------------------------------------------------|-------------------------------------------------------------------------|
| <i>Auth</i>                |                                                    |                                                                         |
| POST                       | <code>/api/v1/auth/register</code>                 | Create user and personal org                                            |
| POST                       | <code>/api/v1/auth/login</code>                    | Get a JWT                                                               |
| <i>Projects</i>            |                                                    |                                                                         |
| POST / GET                 | <code>/api/v1/projects</code>                      | Create / list                                                           |
| GET / PATCH / DELETE       | <code>/api/v1/projects/{id}</code>                 | Read / update / delete (cascades)                                       |
| <i>Queues</i>              |                                                    |                                                                         |
| POST / GET                 | <code>/api/v1/projects/{id}/queues</code>          | Create with config / list                                               |
| GET / PATCH / DELETE       | <code>/api/v1/queues/{id}</code>                   | Read / reconfigure / delete                                             |
| POST                       | <code>/api/v1/queues/{id}/pause, .../resume</code> | Pause / resume consumption                                              |
| GET                        | <code>/api/v1/queues/{id}/stats</code>             | Depth by status, oldest age, durations                                  |
| <i>Jobs</i>                |                                                    |                                                                         |
| POST                       | <code>/api/v1/queues/{id}/jobs</code>              | Immediate or delayed ( <code>delay_s</code> ); idempotency key honoured |
| POST                       | <code>/api/v1/queues/{id}/jobs/batch</code>        | Batch submit in one transaction                                         |
| POST                       | <code>/api/v1/queues/{id}/jobs/retry-failed</code> | Re-enqueue all failed jobs                                              |
| GET                        | <code>/api/v1/jobs</code>                          | List with filters and pagination                                        |
| GET                        | <code>/api/v1/jobs/{id}</code>                     | Detail with executions and logs                                         |
| POST                       | <code>/api/v1/jobs/{id}/cancel</code>              | Cancel (pre-running only)                                               |
| <i>Recurring schedules</i> |                                                    |                                                                         |
| POST / GET                 | <code>/api/v1/queues/{id}/schedules</code>         | Create (cron validated) / list                                          |

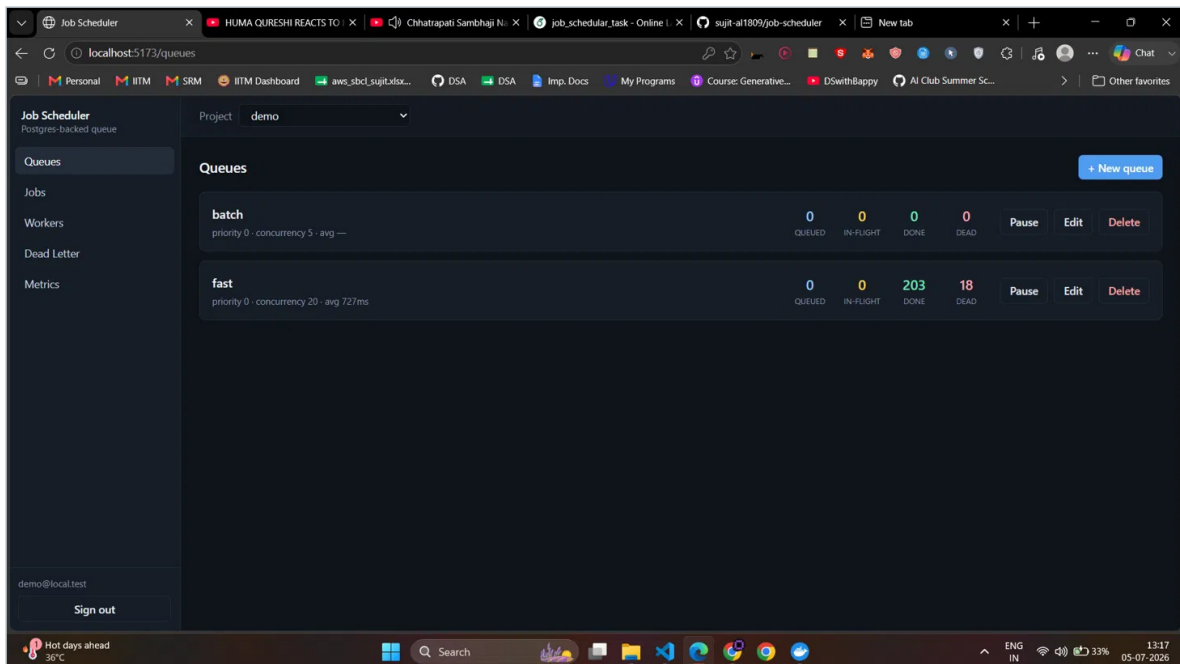
| Method                       | Path                            | Description                                   |
|------------------------------|---------------------------------|-----------------------------------------------|
| PATCH<br>DELETE              | / /api/v1/schedules/{id}        | Pause, edit / remove                          |
| <i>Workers, DLQ, metrics</i> |                                 |                                               |
| GET                          | /api/v1/workers                 | Worker registry with liveness                 |
| GET                          | /api/v1/workers/{id}/heartbeats | Heartbeat history                             |
| GET                          | /api/v1/projects/{id}/dlq       | List dead-letter entries                      |
| POST                         | /api/v1/dlq/{id}/retry          | Re-enqueue with fresh attempts                |
| DELETE                       | /api/v1/dlq/{id}                | Purge an entry                                |
| GET                          | /api/v1/projects/{id}/metrics   | Throughput/min, success rate, p50/p95, depths |
| GET                          | /health                         | Liveness probe, no auth                       |

## 7 Dashboard

The frontend is deliberately plain and operational. React 18 with TypeScript, TanStack Query polling every 5 seconds, Tailwind for styling (responsive layout that holds up down to tablet widths), Recharts for the graphs. Five views: queues (live depth and stats, create/edit config, pause/resume), a job explorer (filterable paginated table, with a detail drawer showing payload, the per-attempt execution timeline, logs, and retry/cancel buttons), workers (liveness, heartbeat age, in-flight sparkline), the DLQ (inspect, re-enqueue, purge), and metrics (throughput per minute, success rate, p95 duration, per-queue depth). Login and registration round it out.

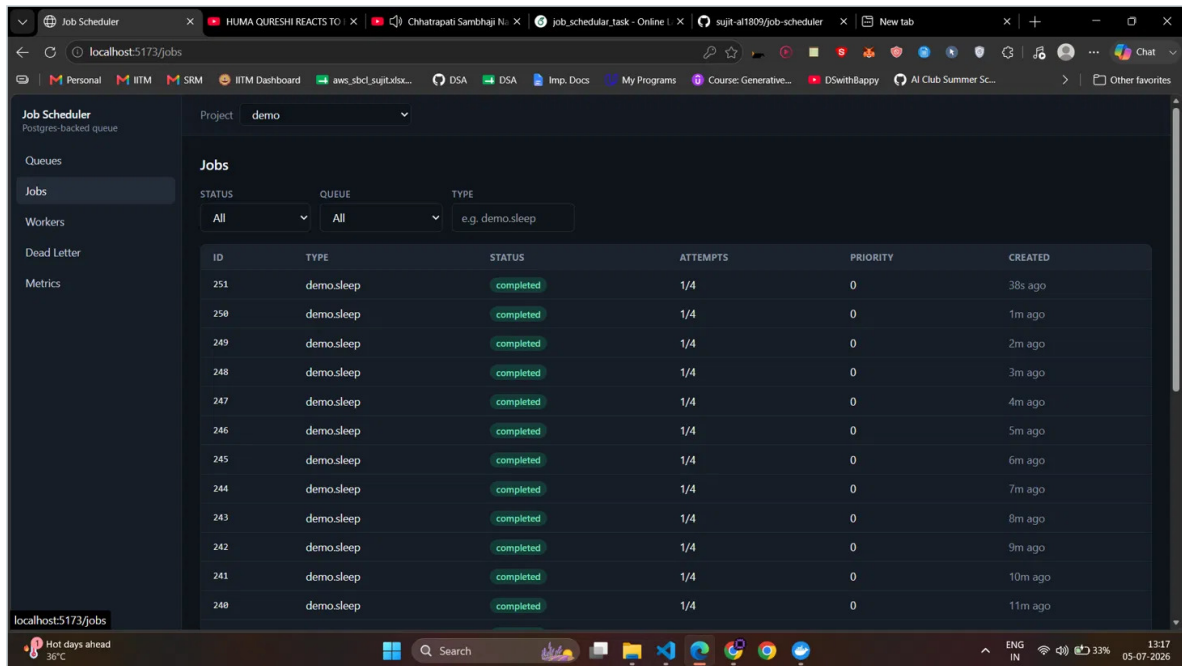
I went with polling instead of WebSockets on purpose — more on that in the next section.

The screenshots below are from a running instance after a demo load (the `demo.sleep`, `demo.random_fail` and `demo.always_fail` handlers), so the numbers are real.

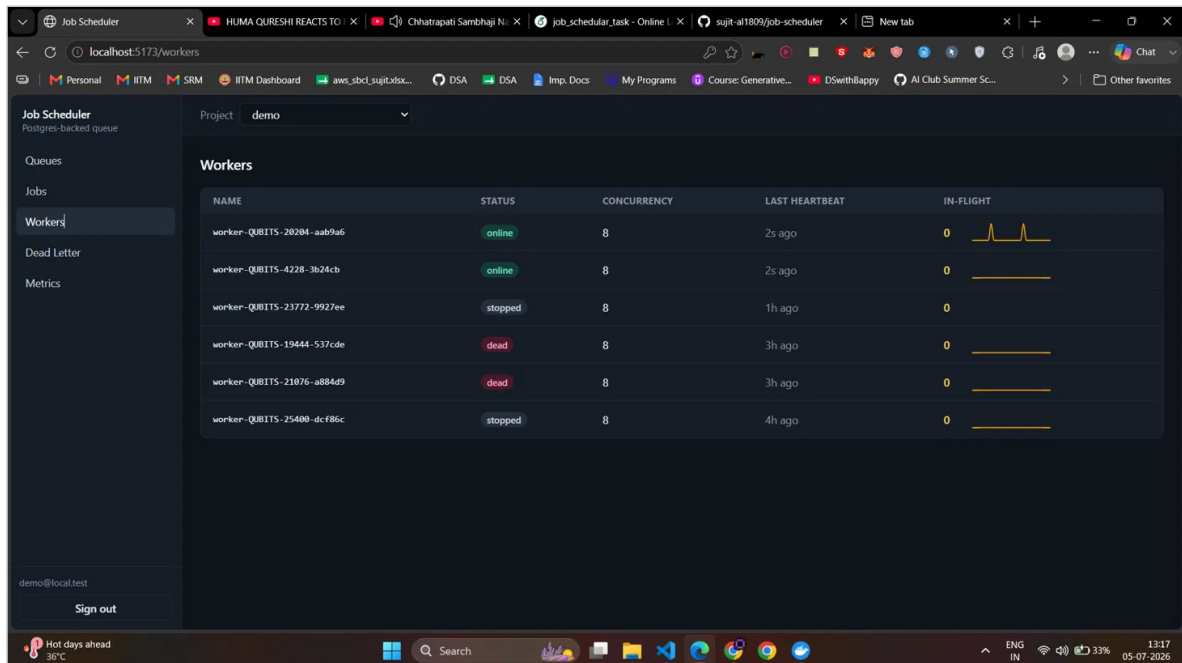


**Figure 2.** Queues view. The fast queue has run 203 jobs to completion with 18 in the dead-letter queue; batch is idle. Each row shows live depth by state and the pause / edit / delete controls.

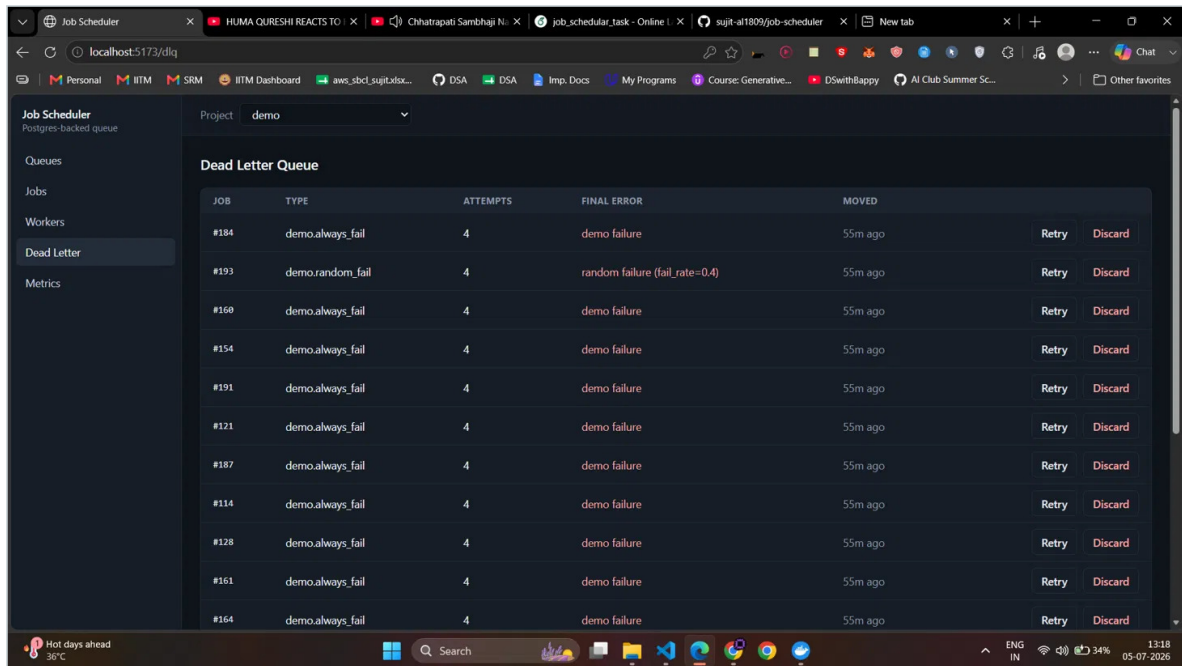




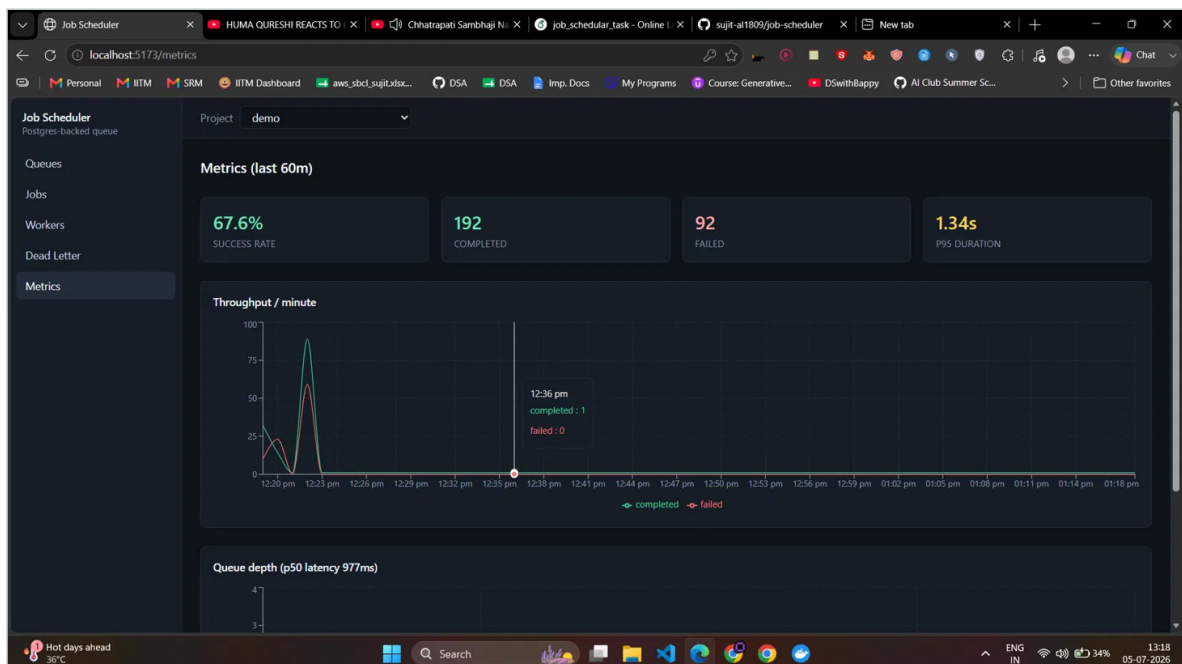
**Figure 3.** Job explorer, filterable by status, queue, and type. Every `demo.sleep` job here completed on its first attempt (1/4), with created-time and priority shown.



**Figure 4.** Worker monitor. Two workers are online and heartbeating (2s ago) with live in-flight sparklines; the others have been marked stopped or dead by the reaper after their heartbeats timed out — the crash-detection path from Section 5.1.



**Figure 5.** Dead-letter queue. Jobs that exhausted all four attempts — `demo.always_fail` and a `demo.random_fail` (`fail_rate=0.4`) — each preserved with its final error and offered for retry or discard.



**Figure 6.** Metrics over the last 60 minutes: success rate, completed / failed counts, and p95 duration up top, with per-minute throughput (completed vs failed) charted below. All derived from `job_executions`.

## 8 Design decisions and trade-offs

**1. Postgres as the queue.** Covered in Section 2.2. The main alternatives were Redis (with something like BullMQ semantics) or RabbitMQ. Both are faster at raw dequeue, but both split the system's state across two stores, and keeping a broker and a database consistent after

crashes is exactly the kind of complexity this assignment does not need. If throughput ever demanded it, the claim logic sits behind a single interface in the worker, so swapping the backing store is contained.

**2. SKIP LOCKED over the alternatives.** I considered advisory locks and optimistic compare-and-swap updates (`UPDATE ... WHERE status = 'queued'` in a loop). Advisory locks move queue semantics into application code and can leak if a connection dies at the wrong moment. CAS loops waste round trips under contention. `SKIP LOCKED` does the partitioning in one statement, and the locks die with the transaction no matter how the process exits.

**3. At-least-once, not exactly-once.** Exactly-once execution would require either holding a transaction open for the entire job run (long transactions, pinned connections — bad) or two-phase commit with every external system a handler touches. Neither is realistic. So the system promises at-least-once and makes redelivery survivable through idempotent handlers. This is also what the mainstream job systems do; I would rather be honest about it and test it than pretend otherwise.

**4. Soft concurrency limits.** Explained in 5.2. A hard limit needs per-queue serialisation of claims; the brief overshoot window is the cheaper problem.

**5. Polling over LISTEN/NOTIFY and WebSockets.** Worker latency is bounded by the poll interval (1 s default) and dashboard freshness by 5 s. Both are fine for background jobs, and the simplicity gain is large. `LISTEN/NOTIFY` to wake idle workers is the first thing I would add later; it would not change the claim path at all.

**6. Retries via `run_at`, never via sleeping.** The wait lives in data, so it costs no worker slot and survives restarts.

**7. A separate scheduler process.** Time-based work is isolated from both the request path and worker capacity. It is also safe to run more than one scheduler, because due cron templates are claimed with `SKIP LOCKED` too — tested with two instances against a one-minute cron.

**8. Database-enforced correctness.** Enums for statuses, a unique index for idempotency, a unique index on (`job_id`, `attempt`). Where the database can make an invalid state impossible, I let it, instead of trusting application code alone.

## 9 Testing

---

Unit tests cover the pure logic: backoff maths, cron advancement, and transition legality. Integration tests run against a real Postgres container — not SQLite, since the whole design leans on `SKIP LOCKED`, partial indexes and `jsonb`, none of which SQLite has.

The tests that carry the reliability claims:

- Four concurrent claim loops over 200 jobs: exactly 200 executions, zero duplicates.
- A soak run: 3 workers, 500 mixed jobs with a 10% random failure rate; every job must end in completed or dead, with attempt counts matching the policy.
- Retry exhaustion: an always-failing job with `max_attempts = 3` produces exactly three execution rows and then lands in the DLQ.
- Crash recovery: `kill -9` a worker mid-job; the reaper requeues it and a second worker finishes it.
- Graceful shutdown: `SIGTERM` with jobs in flight leaves nothing orphaned.

- Scheduler correctness: two scheduler instances, one job per cron tick.
- Ordering and gating: priority order honoured, paused queues never claimed, delayed jobs not claimable early, parallel identical submits with the same idempotency key yield one job.
- Auth and tenancy: cross-org access is 404, missing token is 401.

Concurrency tests have to pass twice in a row in CI. A flaky reliability test is a failing test.

## 10 Setup

Needs Docker with Compose, Python 3.12, and Node 20.

```
git clone <repo-url> && cd distributed-job-scheduler
cp .env.example .env

docker compose up -d db          # Postgres 16 on :5432
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head

uvicorn app.main:app --port 8000  # terminal 1: API (docs at /docs)
python -m scheduler.main         # terminal 2: scheduler + reaper
python -m worker.main            # terminals 3+: run a few workers
cd web && npm install && npm run dev # terminal 4: dashboard on :5173

python scripts/seed_demo.py      # optional demo data
pytest -x                       # tests (db container must be up)
```

Config is all environment variables (see `.env.example`): database URL, JWT secret and TTL, worker concurrency and poll interval, heartbeat interval and timeout, and the shutdown drain timeout.

## 11 Deliverables

Everything the brief asks for, and where it is:

| Deliverable          | Location                                        |
|----------------------|-------------------------------------------------|
| Source code + setup  | the repository; setup in §10 and README.md      |
| Architecture diagram | §2, and docs/architecture.md                    |
| ER diagram           | §3 (Figure 1); DBML source docs/er-diagram.dbml |
| API documentation    | §6, live at /docs, exported docs/openapi.json   |
| Design decisions     | §8, and docs/design-decisions.md                |
| Automated tests      | §9, under tests/                                |

### Bonus features

The brief lists eight optional extras. Where each stands:

| Feature                        | Status                                                                                                       |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| Role-based access control      | Groundwork in place — roles live on <code>org_members</code> ; enforcement is a dependency change (deferred) |
| WebSocket live updates         | Deferred; dashboard uses polling (§8, decision 5)                                                            |
| Rate limiting                  | Deferred; design sketched in §12                                                                             |
| Distributed locking            | Effectively covered for claiming via <code>SKIP LOCKED</code> ; a general lock table is deferred             |
| Workflow dependencies          | Deferred; <code>depends_on_job_id</code> approach noted in §12                                               |
| Queue sharding                 | Deferred (see §8 — low value at this scale)                                                                  |
| Event-driven execution         | Deferred; polling chosen deliberately (§8, decision 5)                                                       |
| AI-generated failure summaries | Deferred; would run over <code>job_logs</code>                                                               |

## 12 If I had more time

Roughly in the order I would do them: LISTEN/NOTIFY so idle workers wake immediately instead of polling; WebSocket push for the dashboard; per-queue rate limiting with a token-bucket table; enforcing the RBAC roles that already exist on `org_members`; and job dependencies (a `depends_on_job_id` column, with children released when the parent completes). I skipped queue sharding and a broker-backed event mode deliberately — at this scale they add operational surface without making anything more correct.